

项目双 Agent 协作体系改造报告

一、任务背景

本项目目前同时使用两个代码 Agent：

- Claude Code
- OpenAI Codex

二者不会在同一时间修改项目。后续希望建立一种明确、稳定、低人工监督成本的协作模式：

- **Claude Code 是唯一的代码实现者。**
- **Codex 是独立的结果审查者。**

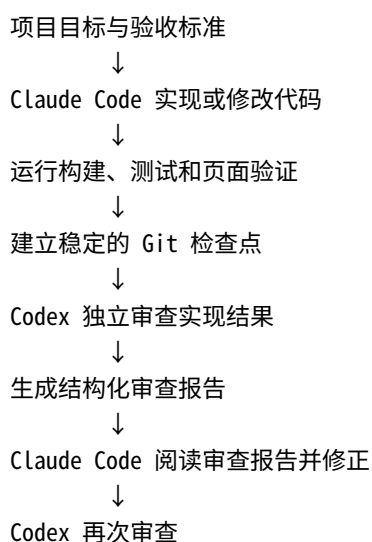
Claude Code 负责阅读目标、编写代码、修改文件、运行测试并完成具体实现。

Codex 不负责直接实现功能，也不应自行修改项目业务代码。Codex 的职责是检查 Claude Code 完成后的结果，发现问题，提供证据，并为下一轮 Claude Code 工作生成明确、可执行、可验证的修改意见。

请 Codex 根据本报告检查当前仓库，并将项目改造为适合这种协作模式的结构。

二、最终协作模型

本项目后续采用以下闭环：



基本原则：

1. Claude Code 负责“做”。
2. Codex 负责“检查”。
3. 两者以仓库中的正式文件进行信息交接。

4. 不依赖临时聊天记录作为项目状态。
 5. 不允许两个 Agent 同时修改业务代码。
 6. 审查意见必须有证据和验收条件。
 7. 审查者不得擅自扩大项目范围。
 8. 项目不能陷入无限修改循环。
-

三、角色边界

3.1 Claude Code：实现者

Claude Code 是项目中唯一被授权修改业务代码的 Agent。

Claude Code 的职责包括：

- 阅读项目目标；
- 阅读最新的 Codex 审查报告；
- 实现项目功能；
- 修复已经确认的问题；
- 运行构建、测试、类型检查、代码检查；
- 必要时启动本地网页并验证页面；
- 生成或更新测试；
- 汇报本轮修改内容；
- 说明尚未解决的问题和风险。

Claude Code 可以修改：

- 源代码；
- 样式；
- 测试；
- 配置文件；
- 构建脚本；
- 项目文档；
- 自动化验证工具。

Claude Code 不得：

- 擅自修改项目最终目标；
 - 删除失败测试以制造“测试通过”；
 - 降低验收标准；
 - 将真实问题简单标记为“无需处理”；
 - 在没有证据的情况下忽略 Codex 提出的 Blocker 或 Major 问题；
 - 擅自扩展与项目目标无关的功能。
-

3.2 Codex：独立审查者

项目完成本次协作体系改造后，Codex 的常规身份是独立审查者。

Codex 的职责包括：

- 阅读项目目标和验收标准；
- 阅读当前实现；
- 审查 Git diff 或指定提交；
- 运行不会破坏项目状态的检查；
- 检查构建、测试、类型检查和 lint 结果；
- 检查主要用户流程；
- 检查响应式布局；
- 检查浏览器控制台错误；
- 检查页面异常状态和边界情况；
- 检查测试是否真正覆盖修改内容；
- 检查实现是否偏离项目目标；
- 生成结构化审查报告。

Codex 在常规审查阶段不得：

- 修改业务代码；
- 修改样式；
- 直接修复问题；
- 顺手重构代码；
- 增加项目目标之外的新功能；
- 用个人审美偏好代替项目验收标准；
- 因为“可以进一步优化”而无限阻止项目完成；
- 同时扮演审查者和实现者。

Codex 可以在审查报告中描述推荐的解决方向，但不能直接替 Claude Code 决定所有实现细节。

四、本次 Codex 的特殊任务

本报告交给 Codex 后，Codex 本轮不是执行常规审查，而是负责完成一次性的“协作体系基础设施改造”。

请首先检查当前仓库结构、技术栈、已有文档、启动命令、测试命令和 Git 状态，然后以最小侵入方式完成改造。

不要机械覆盖已有配置。

如果项目中已经存在功能相同的文件，应优先整合、扩展或复用，而不是重复创建冲突文件。

不得删除当前项目中的有效规则、文档或脚本。

五、需要建立的项目文件

建议最终至少存在以下结构：

```
项目根目录/
├── PROJECT_SPEC.md
├── AGENTS.md
├── CLAUDE.md
├── REVIEW_CONTRACT.md
├── scripts/
│   ├── run-validation.sh
│   ├── run-review.sh
│   └── agent-cycle.sh
└── .agent/
    ├── README.md
    ├── latest-review.md
    ├── implementation-report.md
    ├── state.env
    ├── artifacts/
    └── review-history/
```

请根据现有项目情况调整名称和脚本，但必须保留对应能力。

如果某些生成文件不适合纳入 Git，应同时更新 `.gitignore`。

六、各文件的职责

6.1 PROJECT_SPEC.md

这是项目目标、范围和验收标准的唯一正式来源。

Claude Code 和 Codex 都必须首先阅读该文件。

文件中至少应包含：

项目目标

项目概述

项目是什么，主要服务于什么目标。

最终目标

项目完成后必须达到的结果。

主要用户流程

用户进入网页后能够完成哪些操作。

必须实现

- 功能要求
- 页面要求
- 数据要求

- 交互要求

非目标

本阶段明确不处理的内容。

不允许改变

- 不允许随意改变的技术栈
- 不允许删除的现有功能
- 不允许破坏的页面或数据
- 其他项目约束

验收标准

- 构建要求
- 测试要求
- 页面要求
- 响应式要求
- 浏览器错误要求
- 性能或可访问性要求

项目命令

- 安装依赖
- 启动开发服务器
- 构建
- 测试
- lint
- 类型检查

如果当前项目目标信息不完整，不要凭空设计业务目标。

可以创建结构完整的文件，并将无法从仓库可靠推断的内容标记为：

TODO：由项目所有者补充

不能把 Codex 自己推测的目标写成正式要求。

6.2 CLAUDE.md

该文件用于约束 Claude Code。

建议包含以下规则：

Claude Code 项目规则

你是本项目的唯一代码实现者。

每轮开始前必须阅读：

1. PROJECT_SPEC.md
2. REVIEW_CONTRACT.md
3. .agent/latest-review.md
4. 与当前任务直接相关的代码和测试

你的职责：

- 根据 PROJECT_SPEC.md 实现项目目标；
- 修复审查报告中的有效问题；
- 优先处理 Blocker 和 Major；
- 运行项目规定的验证命令；
- 不得通过删除测试或降低标准制造通过；
- 不得擅自扩展项目范围；
- 不得修改 Codex 的历史审查记录；
- 完成后更新 .agent/implementation-report.md。

每轮结束时必须说明：

- 修改了什么；
- 为什么修改；
- 运行了哪些验证；
- 哪些验证通过；
- 哪些验证失败；
- 是否存在剩余风险；
- 当前提交或 Git 状态。

如果仓库中已有 `CLAUDE.md`，应保留有效内容并进行整合，不能直接覆盖。

6.3 AGENTS.md

该文件用于向 Codex 说明其长期角色。

建议包含：

Codex 项目规则

你是本项目的独立审查者。

常规审查阶段不得修改业务代码。

审查前必须阅读：

1. PROJECT_SPEC.md
2. REVIEW_CONTRACT.md
3. .agent/implementation-report.md
4. 当前 Git diff 或指定提交
5. 相关测试和自动化结果

你的职责：

- 检查实现是否满足项目目标；
- 检查功能错误、回归和遗漏；
- 检查测试是否充分；
- 检查响应式布局 and 主要用户流程；
- 检查构建、类型检查、lint 和控制台错误；
- 为所有问题提供证据；
- 提供可验证的验收条件；
- 输出 `.agent/latest-review.md`。

不得：

- 直接修改业务代码；
- 将个人审美作为阻塞问题；
- 引入 `PROJECT_SPEC.md` 之外的新需求；
- 使用模糊意见，例如“继续优化体验”；
- 在没有证据时判定实现错误；
- 因为 Minor 或 Suggestion 无限阻止项目完成。

如果当前已有 `AGENTS.md`，应整合而不是覆盖。

6.4 REVIEW_CONTRACT.md

该文件规定 Codex 审查报告的格式和判定标准。

建议采用以下严重等级：

Blocker

表示项目无法接受，例如：

- 无法构建；
- 无法启动；
- 核心功能不可用；
- 数据损坏；
- 明确的安全问题；
- 严重回归；
- 关键页面完全不可访问。

Major

表示明显影响项目目标或用户使用，例如：

- 主要功能行为错误；
- 重要边界情况未处理；
- 移动端明显不可用；
- 重复请求；
- 错误状态缺失；

- 测试无法覆盖核心行为；
- 明显偏离 `PROJECT_SPEC.md`。

Minor

表示局部质量问题，例如：

- 非核心样式异常；
- 局部可访问性问题；
- 次要错误提示不清楚；
- 小范围代码可维护性问题。

Suggestion

表示可选优化。

Suggestion 不得阻止项目完成。

审查结论只允许：

VERDICT: PASS

或者：

VERDICT: CHANGES_REQUIRED

建议规定：

- 存在 Blocker：必须 `CHANGES_REQUIRED`；
- 存在 Major：必须 `CHANGES_REQUIRED`；
- 只有 Minor 或 Suggestion：原则上可以 `PASS`；
- 无法验证关键验收条件时，应明确写出验证受阻原因；
- 不能因为无法验证非关键事项而自动判定失败。

七、审查报告格式

`.agent/latest-review.md` 必须使用固定格式。

建议模板如下：

Codex Review

Review metadata

- Reviewed commit:
- Compared against:

- Review date:
- Specification:
- Validation command:
- Scope:

VERDICT: PASS

Executive summary

用简洁语言说明:

- 当前实现是否达到项目目标;
- 是否存在阻塞问题;
- 本轮最重要的发现是什么。

Blocker

如果没有:

None.

如果存在问题:

R-001: 问题标题

- Severity: Blocker
- Requirement: 对应 PROJECT_SPEC.md 中的要求
- Evidence: 可复现证据
- Location: 文件、组件、路由或功能位置
- Impact: 对用户或项目的影响
- Reproduction:
 1. 第一步
 2. 第二步
 3. 观察结果
- Expected: 正确结果
- Actual: 当前结果
- Acceptance criteria: Claude Code 修复后如何验证
- Suggested direction: 可选的处理方向

Major

使用相同结构。

Minor

使用相同结构。

Suggestions

使用相同结构，但明确说明不阻止通过。

Validation results

- Dependency installation:
- Build:
- Tests:
- Lint:
- Type check:
- Browser console:
- Responsive checks:
- Main user flow:
- Other:

Confirmed working

列出已经验证通过的内容。

Unverified areas

列出由于环境、凭据、网络、浏览器或其他原因无法验证的内容。

Required next actions

只列出下一轮 Claude Code 必须执行的事项。

按以下顺序排列：

1. Blocker
2. Major
3. 必要的验证工作

不要把 Suggestion 混入必须执行事项。

每个问题必须具备以下内容：

1. 明确标题；
2. 严重程度；
3. 对应要求；
4. 证据；
5. 影响；
6. 复现方式；
7. 预期结果；
8. 实际结果；
9. 可验证的验收条件。

禁止输出以下类型的模糊意见：

- “用户体验可以进一步提高”；
- “代码还可以更优雅”；
- “页面还可以更现代”；
- “建议继续重构”；
- “最好增加更多功能”。

除非这些意见能够关联具体目标、具体证据和具体验收条件。

八、实现者报告格式

`.agent/implementation-report.md` 由 Claude Code 每轮更新。

建议模板：

Claude Code Implementation Report

Implementation metadata

- Base commit:
- Result commit:
- Date:
- Review addressed:
- Working tree status:

Objective

本轮要完成的目标。

Changes made

C-001: 修改标题

- Related review item:
- Files changed:
- Implementation:
- Reason:

Review items addressed

- R-001: Fixed
- R-002: Fixed
- R-003: Not fixed

对于未修复的问题，必须解释原因。

Validation performed

- Build:
- Tests:
- Lint:
- Type check:
- Browser checks:
- Screenshots:
- Other:

Remaining issues

列出已知但未解决的问题。

Risks

列出可能存在的回归或不确定性。

Handoff to Codex

说明 Codex 下一轮需要重点检查的内容。

九、自动验证脚本

9.1 `scripts/run-validation.sh`

请根据项目实际技术栈创建统一验证入口。

脚本职责应包括：

- 检查依赖是否存在；
- 运行构建；
- 运行测试；
- 运行 lint；
- 运行类型检查；
- 必要时运行网页自动化测试；
- 保存完整日志；
- 失败时返回非零退出码。

建议输出目录：

```
.agent/artifacts/validation/
```

可能的结果文件：

```
.agent/artifacts/validation/build.log
.agent/artifacts/validation/tests.log
.agent/artifacts/validation/lint.log
.agent/artifacts/validation/typecheck.log
.agent/artifacts/validation/summary.md
```

脚本必须适配项目已有的包管理器和命令。

不要默认项目一定使用 npm。

应检查并识别：

- package-lock.json
- pnpm-lock.yaml
- yarn.lock
- bun.lock
- bun.lockb

同时读取 package.json 中已有的 scripts。

不能为了建立统一命令而破坏现有工作流。

如果某项检查当前不存在，应在总结中标记为：

NOT CONFIGURED

不能把“不存在测试”伪装为“测试通过”。

9.2 网页视觉和行为验证

如果项目是可运行的网页项目，应尽可能复用或建立 Playwright 等浏览器自动化能力。

至少考虑验证：

- 首页能够打开；
- 主要路由能够打开；
- 核心用户流程能够执行；
- 无未处理的浏览器控制台错误；
- 无明显页面崩溃；
- 无横向溢出；
- 主要按钮可点击；
- 请求期间状态合理；
- 错误状态能够显示；
- 404 或无效路径行为合理。

建议至少使用以下视口：

390 × 844
768 × 1024
1440 × 900

建议输出：

.agent/artifacts/screenshots/mobile/
.agent/artifacts/screenshots/tablet/
.agent/artifacts/screenshots/desktop/

```
.agent/artifacts/browser/console-errors.txt
.agent/artifacts/browser/network-errors.txt
.agent/artifacts/browser/test-results/
```

只有在适合当前项目且不会引入过重依赖时，才添加新的浏览器测试框架。

如果项目已有 Cypress、Playwright、Vitest Browser Mode 或其他工具，应优先复用。

十、Codex 审查脚本

10.1 `scripts/run-review.sh`

创建一个用于启动 Codex 审查的脚本。

脚本应做到：

1. 检查 Git 工作区状态；
2. 确定要审查的提交或 diff；
3. 运行统一验证脚本，或者读取最近一次可靠验证结果；
4. 启动 Codex 的只读审查；
5. 将最终审查报告写入 `.agent/latest-review.md`；
6. 将报告归档到 `.agent/review-history/`；
7. 不修改业务代码；
8. 审查失败时保留错误日志；
9. 使用非零退出码表示审查流程本身失败。

推荐使用只读沙箱。

Codex 的审查提示应强调：

你是独立审查者，不是实现者。

不得编辑业务代码。

必须根据 `PROJECT_SPEC.md`、Git diff、实现者报告和验证证据进行审查。

不得增加项目范围。

必须将所有问题写成可执行、可验证的审查条目。

最终输出必须符合 `REVIEW_CONTRACT.md`。

脚本不能依赖 Codex 自己无限递归调用 Codex。

十一、协作循环脚本

11.1 `scripts/agent-cycle.sh`

可以建立一个半自动协作脚本，但不要默认启动完全无人监管、无限运行的循环。

脚本建议支持以下模式：

```
./scripts/agent-cycle.sh validate
./scripts/agent-cycle.sh review
./scripts/agent-cycle.sh status
./scripts/agent-cycle.sh archive
```

如果当前环境能够稳定支持 Claude Code 的非交互执行，也可以增加：

```
./scripts/agent-cycle.sh implement
./scripts/agent-cycle.sh cycle
```

但必须具备安全边界：

- 默认最多运行有限轮数；
- 建议最大三轮；
- 每轮都需要明确的 Git 边界；
- 不允许两个 Agent 同时运行；
- 不允许审查者修改业务代码；
- 工作区不干净时不得执行危险的 reset；
- 不得自动删除用户未提交修改；
- 不得自动强制推送；
- 不得自动向远程仓库推送；
- 不得自动合并到主分支；
- 不得自动覆盖项目所有者的修改；
- 不得无限重试；
- 不得自动购买或消耗额外付费额度。

推荐流程：

1. 检查环境
2. 检查 Git 状态
3. Claude Code 实现
4. 运行统一验证
5. 建立或要求建立检查点
6. Codex 只读审查
7. 写入最新审查报告
8. 如果 PASS，则结束
9. 如果 CHANGES_REQUIRED，则进入下一轮
10. 达到最大轮数后停止，并交由项目所有者决定

如果无法可靠自动调用 Claude Code，不要伪造该能力。

应保留清晰的手动命令，让用户可以：

1. 手动运行 Claude Code；
2. Claude 完成后运行 `run-validation.sh`；
3. 再运行 `run-review.sh`。

十二、Git 协作规则

Git 是两个 Agent 之间的稳定边界。

请建立以下规则：

1. Claude Code 修改前读取当前 Git 状态。
2. Claude Code 不得覆盖用户已有的未提交修改。
3. Codex 优先审查明确提交或明确 diff。
4. 审查报告中记录：
5. 被审查提交；
6. 对比基准；
7. 工作区是否干净。
8. 审查者不得使用破坏性 Git 命令。
9. 自动化脚本不得执行：
10. `git reset --hard`
11. `git clean -fd`
12. 强制 checkout
13. 强制 push
14. 未经许可的 rebase
15. 如工作区存在未提交修改，应停止并报告，而不是擅自处理。

建议提交信息格式：

```
agent: implementation round 1
agent: address review R-001 R-002
agent: collaboration infrastructure
```

如果项目已有提交规范，应优先遵守现有规范。

十三、审查隔离

Codex 的常规审查应尽可能采用技术隔离，而不是只依靠自然语言。

优先级如下：

1. Codex 使用只读沙箱；
2. 审查明确提交而不是混乱的工作区；
3. 必要时使用独立 Git worktree；
4. Codex 只写入允许的报告和日志路径；
5. Codex 不获得业务代码写权限。

如果需要创建独立 worktree，请提供安全脚本或清晰说明，但不要在工作区不明时自动执行危险操作。

推荐目录示意：

```
~/projects/project  
~/projects/project-review
```

其中：

- `project`：Claude Code 实现；
- `project-review`：Codex 只读审查。

不过，只有在确实能提高安全性且不会使当前项目过度复杂时才采用 worktree。

十四、项目状态文件

建议创建 `.agent/state.env`，用于记录机器可读状态。

示例：

```
CURRENT_ROUND=0  
LAST_IMPLEMENTATION_COMMIT=  
LAST_REVIEWED_COMMIT=  
LAST_REVIEW_VERDICT=  
LAST_VALIDATION_STATUS=  
MAX_ROUNDS=3
```

脚本更新该文件时必须：

- 保持格式简单；
- 不存储密钥；
- 不存储访问令牌；
- 不存储个人敏感信息；
- 不将临时错误状态误写成永久项目状态。

十五、归档规则

每次新的 Codex 审查完成后：

1. 更新 `.agent/latest-review.md`；
2. 将同一份报告复制到：

`.agent/review-history/`

推荐文件名：

`2026-07-22_round-01_<commit>.md`

或者使用项目当前时间动态生成。

Claude Code 不得修改历史审查文件。

Codex 可以在新报告中说明旧问题已经解决，但不应重写历史。

验证日志和截图可以根据体积决定是否纳入 Git。

应在 `.agent/README.md` 中说明：

- 哪些文件需要提交；
- 哪些文件是本地生成；
- 哪些目录被 `.gitignore` 忽略。

十六、终止条件

为了防止两个 Agent 无限互相修改，项目必须有明确终止规则。

建议：

可以通过

满足以下条件时，Codex 可以输出：

`VERDICT: PASS`

条件：

- 不存在 Blocker；

- 不存在 Major;
- 核心验收条件通过;
- 构建成功;
- 关键测试通过;
- 主要用户流程可用;
- 没有已确认的严重回归。

即使存在少量 Minor 或 Suggestion, 也允许通过, 但必须在报告中记录。

必须继续修改

存在以下情况时输出:

VERDICT: CHANGES_REQUIRED

- 存在 Blocker;
- 存在 Major;
- 核心目标未实现;
- 构建失败;
- 关键流程不可用;
- 实现明显偏离 `PROJECT_SPEC.md`。

强制停止并交还用户

出现以下情况时不要继续自动循环:

- 达到最大轮数;
- 两轮连续出现相同问题;
- 需要改变项目目标;
- 需要用户做产品决策;
- 需要凭据、付款或外部授权;
- 自动化环境无法可靠验证;
- Claude Code 和 Codex 对要求存在根本冲突;
- Git 工作区状态不安全;
- 可能覆盖用户修改。

十七、范围控制

Codex 必须将问题与 `PROJECT_SPEC.md` 关联。

以下不应自动成为阻塞事项:

- 个人审美偏好;
- 未要求的动画;
- 未要求的暗色模式;
- 未要求的账户系统;
- 未要求的国际化;

- 未要求的框架迁移；
- 仅仅因为另一种写法更优雅而进行重构；
- 与当前目标无关的性能微优化。

如果 Codex 发现有价值但不在范围内的改进，可以放入：

Suggestions

不得放入：

Required next actions

十八、安全要求

本次改造和后续自动化必须遵守：

1. 不删除用户文件。
2. 不覆盖用户未提交修改。
3. 不修改系统文件。
4. 不读取或输出密钥。
5. 不将 `.env` 内容写入报告。
6. 不打印访问令牌。
7. 不自动推送远程仓库。
8. 不自动部署生产环境。
9. 不自动修改线上数据。
10. 不自动执行不可逆迁移。
11. 不自动运行高风险数据库命令。
12. 不使用无限权限作为默认方案。
13. 审查过程原则上只读。
14. 对可能破坏状态的命令先停止并说明。

十九、本次改造的实施步骤

请 Codex 按以下顺序执行：

第一步：仓库调查

检查并记录：

- 当前技术栈；
- 包管理器；
- 项目启动方式；
- 构建命令；
- 测试命令；

- lint 命令；
- 类型检查命令；
- 当前 Git 状态；
- 当前已有的 AGENTS.md ；
- 当前已有的 CLAUDE.md ；
- 当前文档结构；
- 当前自动化测试能力；
- 当前浏览器测试能力；
- 当前 CI 配置。

第二步：提出内部改造方案

根据实际仓库进行最小侵入设计。

不要为了符合本报告而引入明显不必要的复杂度。

第三步：创建或整合协作文件

完成：

- PROJECT_SPEC.md
- AGENTS.md
- CLAUDE.md
- REVIEW_CONTRACT.md
- .agent/README.md
- .agent/latest-review.md
- .agent/implementation-report.md
- .agent/state.env

第四步：创建验证和审查脚本

完成：

- 统一验证脚本；
- Codex 只读审查脚本；
- 必要的状态或归档脚本；
- 可选的有限循环脚本。

第五步：配置忽略规则

根据实际情况更新 .gitignore 。

不要忽略应该纳入版本控制的核心协议文件。

第六步：执行验证

至少验证：

- 脚本语法；

- 路径正确；
- 命令能够找到；
- 验证脚本能够运行；
- 报告输出目录能够生成；
- 脚本失败时返回正确退出码；
- 不会删除或覆盖用户修改。

第七步：生成改造总结

完成后输出一份总结，包括：

- 新增文件；
- 修改文件；
- 每个文件的职责；
- 推荐的日常使用流程；
- 当前尚未配置的能力；
- 需要用户补充的 `PROJECT_SPEC.md` 内容；
- 使用 Claude Code 实现一轮的命令；
- 使用 Codex 审查一轮的命令；
- 已知限制和风险。

二十、本次改造验收标准

本次任务完成的标准如下：

- ☐ 仓库中存在明确的唯一目标文件。
 - ☐ Claude Code 的实现者身份得到明确约束。
 - ☐ Codex 的审查者身份得到明确约束。
 - ☐ Codex 常规审查时不修改业务代码。
 - ☐ 存在正式的审查报告契约。
 - ☐ 审查问题具有严重程度、证据和验收条件。
 - ☐ 存在统一验证入口。
 - ☐ 存在 Codex 只读审查入口。
 - ☐ 存在最新审查报告路径。
 - ☐ 存在历史审查归档机制。
 - ☐ 存在 Claude Code 实现报告路径。
 - ☐ 存在最大循环轮数或其他终止规则。
 - ☐ 自动化不会删除未提交修改。
 - ☐ 自动化不会自动推送远程仓库。
 - ☐ 自动化不会无限运行。
 - ☐ 缺失的项会被标记，而不是伪装为通过。
 - ☐ 项目所有者能够用少量命令完成一轮“实现—验证—审查”流程。
 - ☐ 本次改造不破坏项目现有启动、构建和测试流程。
-

二十一、推荐的日常工作方式

完成改造后，项目所有者应能够按照类似方式工作。

Claude Code 实现阶段

向 Claude Code 提供：

阅读 PROJECT_SPEC.md、CLAUDE.md、REVIEW_CONTRACT.md 和
.agent/latest-review.md。

你是本项目的唯一实现者。

完成当前项目目标，并修复最新审查报告中的所有 Blocker 和 Major。
运行项目规定的验证命令。

不得降低验收标准，不得删除失败测试，不得擅自增加项目范围。

完成后更新 .agent/implementation-report.md，并停止。

验证阶段

运行：

```
./scripts/run-validation.sh
```

Codex 审查阶段

运行：

```
./scripts/run-review.sh
```

下一轮

Claude Code 再次读取：

```
.agent/latest-review.md
```

然后继续修复。

二十二、Codex 本次执行要求

请现在开始检查当前仓库，并完成本报告要求的协作体系改造。

执行时遵守以下优先级：

1. 保护现有项目和用户修改；
2. 适配项目当前技术栈；
3. 建立清晰的角色边界；
4. 建立可靠的信息交接文件；
5. 建立可验证的审查流程；
6. 尽量降低项目所有者的监督成本；
7. 避免过度工程化。

本轮允许你修改仓库，因为本轮任务就是建立协作基础设施。

但是从本轮改造完成以后，你的默认角色应恢复为：

只负责检查 Claude Code 的实现结果，并提出下一轮修改意见，不直接修改业务代码。

完成后不要立即开始审查或重写整个项目。

先向项目所有者汇报：

- 你具体做了哪些改造；
- 如何启动 Claude Code 实现轮次；
- 如何启动 Codex 审查轮次；
- 哪些项目目标仍需项目所有者补充；
- 当前工作区和 Git 状态。